

Evaluating the Effectiveness of Algorithm Analysis Visualizations

Mohammed F. Farghally*, Kyu Han Koh†, Hossameldin Shahin*,
and Clifford A. Shaffer*

*mfseddik,hshahin,shaffer@vt.edu, Dept. of Computer Science, Virginia Tech, Blacksburg, VA 24061

†kkoh@csustan.edu, Dept. of Computer Science, CSU–Stanislaus, Turlock, CA 95382

ABSTRACT

Algorithm Visualizations (AVs) have been used for years as an interactive method to convey data structures and algorithms concepts. However, AVs have traditionally focused on illustrating the mechanics of how an algorithm works. We have developed visualizations that we name *Algorithm Analysis Visualizations* (AAVs), that focus on conveying algorithm analysis concepts. We present our findings from an initial evaluation study of the effectiveness of AAVs when applied to a semester long Data Structures course. AAVs were evaluated in terms of student engagement, student satisfaction, and student performance. Results indicate that the intervention group students spent significantly more time with the AAVs than did the control group students who used primarily textual content. Students gave positive feedback regarding the usefulness of the AAVs in illustrating algorithm analysis concepts. Students from the intervention group had better performance on the algorithm analysis part of the final exam than did control group students.

Keywords

Algorithm Analysis Visualizations; Log Data Analysis; Performance Evaluation

1. INTRODUCTION

Data Structures and Algorithms (DSA) are fundamental topics in CS curricula. DSA courses include concepts related to algorithm dynamics (how an algorithm works) and algorithm analysis (algorithm efficiency). DSA concepts are traditionally hard for many students to grasp as they are abstract and mathematical in nature [2]. Accordingly, special attention must be given to the ways that these topics are presented to students. Better techniques that improve students' understanding in this area could have high impact.

Algorithm Visualizations (AVs) have been a part of the DSA curriculum since 1981 when Roland Baecker introduced the “Sorting out Sorting” video. AVs typically include an illustration of the step-by-step execution of algorithms and how each step affects the state of the underlying data structure [16]. There has long been a belief that AVs will be pedagogically effective when used in DSA courses. However, the DSA community has encountered mixed results when evaluating the effectiveness of AVs [11].

Algorithm analysis concepts are crucial in any DSA course. Students are expected to understand each algorithm in terms of resource requirements in different situations. But analysis concepts are often not well understood by many students, perhaps because these concepts are presented and justified by invoking sophisticated mathematical arguments that most CS students have trouble understanding [7, 15]. To our knowledge, few AVs focus on algorithm analysis concepts [16]. Hence AV developers are missing one of the most important concepts in any DSA course.

A previous study [5] found that algorithm analysis topics are among the hardest topics in a typical post-CS2 DSA course — as indicated from student scores on eTextbook exercises, item response theory analysis, and instructor surveys. The eTextbook used in the study was the main textbook in the course. It originally presented algorithm analysis material relying mainly on textual discussion supported by static images, similar to a standard textbook. In order to see whether this material was effective for students, student interaction logs were analyzed from use of the eTextbook's analysis material related to sorting (this chapter contains more than 40% of all analysis material in the book). By investigating student interaction logs from three different universities offering the same course, it was found that over 74% of the students spent less than one minute on the analysis material for a given sorting module. Based on this result, we believe that most of the students do not read the analysis material. Surveys indicate that instructors find algorithm analysis topics among the hardest both for them to teach and for students to learn.

Our response was to develop a more engaging way to present algorithm analysis topics. We have created a pool of Algorithm Analysis Visualizations (AAVs) capable of interactively conveying algorithm analysis concepts presented in a typical DSA course.

We present results from an evaluation designed to gauge the effectiveness of AAVs in terms of student engagement and exam performance. The AAVs that we developed were deployed as part of the open-source OpenDSA eTextbook

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGCSE'17 March 8-11, 2017, Seattle, USA

© 2017 ACM. ISBN 978-1-4503-4698-6/17/03...\$15.00

DOI: <http://dx.doi.org/10.1145/3017680.3017698>

project [4, 6]. OpenDSA provides infrastructure and content necessary to create a collection of online, open-source tutorials that combine textbook-quality text with visualizations and exercises with automated feedback. Our AAVs were embedded in the eTextbook that was used as the primary content for algorithm analysis in the DSA course.

This paper is organized as follows. We present background on the use of visual proofs in Section 2, and the concept of AAVs in Section 3. Our evaluation protocol, along with results on student engagement, student satisfaction, and student performance are presented in Sections 4 and 5. We discuss possible threats to the validity of the study in Section 6, and future work plans in Section 7.

2. RELATED WORK

For an overview of general AV history, refer to [16]. Here we focus on prior attempts to present algorithm analysis topics in a visual manner. To our knowledge there is no support in the literature for visualizing basic algorithm analysis concepts such as running-time complexity, growth rates, and upper or lower bounds. Presenting such concepts visually is a challenge, because unlike algorithm dynamics, algorithm analysis has no dynamic process to visualize. Instead, we can attempt to use visual features in explanatory material such as proofs.

Visual proofs is an approach to presenting proofs visually by augmenting or even replacing inductive arguments that most students find difficult. Goodrich and Tamassia [7] present simple visual proofs for several core topics in DSA courses (summing linear terms, counting nodes in a binary tree, analyzing binary tree traversal, analyzing bottom-up heap construction, rebalancing AVL trees via rotations) in an attempt to justify the potential of using visual alternatives for teaching algorithm analysis concepts. Thompson and Chadhuri [18] present an alternative visual analysis of the Build-heap algorithm (as previously presented in [7]). Blaheta [1] presents a visual proof for amortized-linear resizable arrays by proving that doubling the capacity is the best strategy when resizing linear arrays. Sher [17] presents a visual proof of the average case running time of a list-searching algorithm. Hammack and Lyons [9] provide a picture proof of the alternating series test using simple comparisons of areas of rectangles to establish convergence.

We find that visual proofs can be characterized as making use of one or both of two major presentation principles: the “Area-to-Cost Principle” and the “Visual Description Principle”. The “Area-to-Cost Principle” uses graphical primitives to represent the amount of work required for each algorithm step, and then the total running time for the algorithm can be viewed as the total surface area of the resulting shape. This approach was applied in the Build-heap visual proof [18], the visual proof to find the closed form solution of the summation $\sum_{i=1}^n i$ presented in [7], the amortized-linear resizable array proof [1], and the alternating series convergence proof [9]. Figure 1 shows the Build-heap visual proof as presented in [18]. On the other hand, the binary tree traversal, the bottom-up heap construction description, and the analysis of AVL trees proofs presented in [7] apply what we refer to as the “Visual Description Principle”. They present a series of visual depictions that support the (otherwise textual) discussion step by step. The proof mainly relies on the intuition provided by the visual depictions associated with the text. Figure 2 shows the binary tree traversal

visual proof as presented in [7]. The Visual Description Principle embodies the application of the Multimedia Principle of Mayer [14] to AAVs.

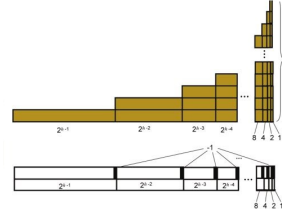


Figure 1: Buildheap visual proof [18]

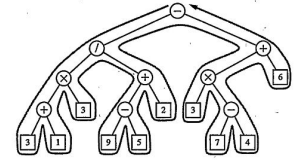


Figure 2: Binary tree traversal visual proof [7]

To our knowledge, there has been no attempt to present algorithm analysis concepts using interactive visualizations that can be integrated into the curriculum, as is done with AVs. The visual proofs literature does not include any pedagogical evaluation to determine whether they are engaging or useful for student learning.

3. AAV IMPLEMENTATION

Motivated by the findings that students were having difficulty with algorithm analysis topics, and that at the same time they were not spending sufficient time reading the text-based material, we decided to create a pool of AAVs to replace the text-based content. The AAVs were developed using the JSAV framework [12], a JavaScript library that makes the AAVs easy to embed within HTML pages. So far, we have developed 28 AAVs. 14 are related to the running time of sorting algorithms, and the other 14 are related to general introductory algorithm analysis concepts. The AAVs were integrated into the OpenDSA eTextbook system to present the algorithm analysis material available in the sorting and algorithm analysis introduction chapters.

Figures 3 and 4 show two representative AAVs. Each is composed of a series of slides, and each slide has both visual and textual components. Navigation through the slides is controlled by forward and backward buttons, with the option to go directly to the first or last slide. So the viewer controls pacing through the material. The textual component on each slide is a brief statement, supported by the visual component.

Figure 3 shows the AAV for the worst-case analysis of Insertion Sort. This AAV relies on the Area-to-Cost Principle. An input array that produces a worst-case problem instance is displayed. A code display is shown to relate each algorithm step with the line of code that produces it. The idea here is to show the step by step execution of the algorithm on the worst-case instance presented, and at each step show the amount of work that this single step produces, which is here shown as a single rectangle. At the end we reach the shape shown in the figure, and the total amount of work will correspond to the surface area of this shape, which can be easily calculated from the figure by counting the rectangles as $\frac{(n-1)(n-1)}{2} + \frac{(n-1)}{2} = \frac{n(n-1)}{2}$. We know that the worst-case running time of Insertion Sort is described by the summation $\sum_{i=1}^{n-1} i$, where n is the size of the array. This summation is evaluated to $\frac{n(n-1)}{2}$ which is the same as the total surface area of the shape shown in the figure.

Figure 4 presents the sorting lower bound proof, and is an

example of an AAV developed using the Visual Description Principle. The proof is presented step by step, by gradually creating the decision tree that models the processing of the Insertion Sort algorithm on an array of 3 elements.

4. EVALUATION PROTOCOL

We have three questions about the use of AAVs that we sought to answer through various evaluation efforts. (1) Do students in the intervention group engage more with AAVs than the control group students engaged with traditional algorithm analysis content? (2) What feedback do students from the intervention group give regarding their experience with AAVs? (3) How does the performance of the intervention group on the algorithm analysis part of their final exam compare to the performance of the control group students on the same set of questions?

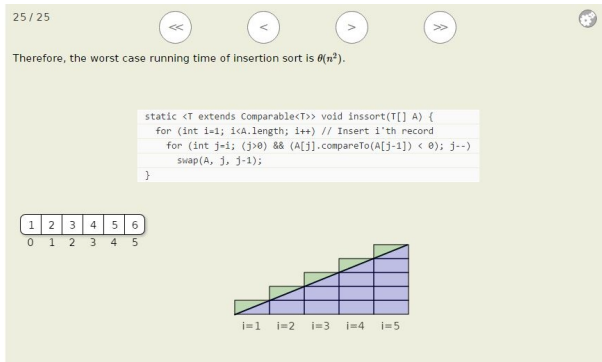


Figure 3: Insertion Sort worst-case runtime analysis

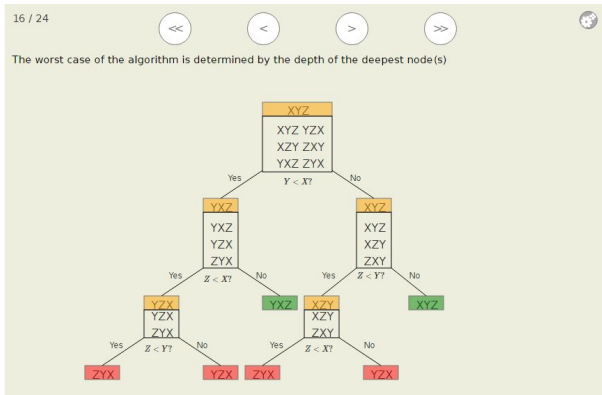


Figure 4: Sorting lower bound proof

Our control group was 67 students taking a DSA course at a large research university during Fall 2015. During Spring 2016, 155 students taking the same DSA course served as our intervention group. Both the intervention and control groups used OpenDSA as their main textbook. Our intervention focused on the presentation of algorithm analysis material in the sorting and the algorithm analysis introduction chapters. The control group saw a textual presentation of this content, like in a traditional DSA textbook. For the intervention group, much of this content was presented interactively using AAVs. The algorithm dynamics concepts were presented visually in both groups using traditional AVs.

The total time spent on algorithm analysis content (text-based content for the control group, or an AAV for the in-

tervention group) was used as a proxy for engagement. The total time was calculated by analyzing OpenDSA student interaction logs.

In order to evaluate AAVs in terms of student learning, both groups were given a set of algorithm analysis questions as part of their final exam. The algorithm analysis part of the test for Fall 2015 was composed of 9 items with some divided into sub-items for a total of 28 items. A similar test was offered in Spring 2016 with 11 items, some divided into sub-items for a total of 29 items. 27 items were similar in both tests. These items form the basis of our performance comparison between both groups. Exams were graded by the same person for both groups to minimize inter-rater reliability problems.

In order to investigate the students' opinions about AAVs, we offered a survey at the end of Spring 2016 asking students to evaluate AAVs in terms of how useful they were in helping them understanding the algorithm analysis concepts presented in the course.

We decided to use non-parametric tests (Mann-Whitney) with 5% significance levels to analyze data comparing the control and intervention groups for time spent and exam grade. We chose this test to avoid the normality assumption required if we would apply a parametric test, and to mitigate the effect of unbalanced sample sizes [13], as there were more students during Spring 2016.

5. EVALUATION RESULTS

5.1 Student Engagement

Time spent is an accepted measure to assess student engagement in learning [8]. OpenDSA interaction logs for the control group were analyzed to find an estimate of the total time each student spent in the text-based algorithm analysis material in the sorting modules. Sorting modules generally start with a detailed discussion of how a given sorting algorithm works, and then the running time analysis of the algorithm is presented. To separate the time spent on analysis topics from the rest of the module content, we made the algorithm analysis material viewable only after clicking a button. Total time was calculated as the sum of the times a student spent viewing algorithm analysis content, since a student might view the same content multiple times. We define a reading session as the difference between the time when the button to show the algorithm analysis content is pressed and the time when the student performs any other interaction with the module (i.e., started solving an exercise, left the module, refreshed the page). Similarly, for the intervention group in Spring 2016, OpenDSA interaction logs were analyzed to estimate the total time a student spent using the corresponding AAV. Again, the total time was calculated as the sum of the times the student spent in each session. We define this as the difference in times between the first and last button presses associated with the AAV.

We did the same for OpenDSA algorithm analysis introductory modules. However, as all of the content in these modules is related to algorithm analysis, we simply summed any sessions spent in the module. For Fall 2015 students, the modules were presented without AAVs, and for Spring 2016, the modules were enhanced with some AAVs for part of the discussion. We define a module session as the difference between the time the student loaded the module and the time the student leaves the module.

In a few cases, we calculated large times, which we don't believe to be valid estimates of time spent. Our explanation for these cases is that the student started to interact with the material (the starting time is recorded) and then left the browser open while doing something else. After a relatively long time the student came back and interacted again with the material until they were done (the end time is recorded). In order to perform valid analysis, we discarded all sessions with times identified as outliers based on a simple outlier analysis (using the JMP software package).

After calculating the time for both groups, we compared the total time spent for each student. Results are presented in Table 1 for sorting modules and in Table 2 for algorithm analysis introduction modules. The “exchange sorting cost” and the “sorting lower bound” modules are part of the sorting chapter, but their content is entirely related to algorithm analysis. Accordingly, the time spent using them was calculated in the same way as the algorithm analysis introduction modules. In the tables below, N means the number of students who interacted with the algorithm analysis material presented in the given modules. F15 denotes the Fall 2015 (control) group, and S16 denotes the Spring 2016 (intervention) group. The mean and median times are in seconds.

Table 1: Results for the sorting modules.

Module	Group	N	Mean	Median	p
Insertionsort	F15	30	76.77	13.5	< 0.0001
	S16	123	189.26	127	
Bubblesort	F15	28	23.57	12	< 0.0001
	S16	137	78.96	52	
Selectionsort	F15	22	37.81	21.5	< 0.0001
	S16	135	104.14	76	
Mergesort	F15	28	23	5.5	< 0.0001
	S16	134	93.02	60	
Quicksort	F15	28	76.64	14.5	< 0.0001
	S16	133	194.98	122	
Heapsort	F15	26	20.88	6.5	< 0.0001
	S16	121	95.22	46	
Radixsort	F15	21	25.23	4	< 0.0001
	S16	123	135.63	52	
Exchange Sorting cost	F15	29	148.03	42	< 0.0001
	S16	117	610.59	438	
Sorting Lower Bound	F15	27	78.55	17	< 0.0001
	S16	103	610.41	495	

Table 2: Results for analysis intro modules.

Module	Group	N	Mean	Median	p
Asymptotic Analysis and Upper Bounds	F15	35	181.65	56	< 0.0001
	S16	122	970.95	649	
Best, Average, and Worst Cases	F15	38	215.8	38	0.0037
	S16	88	412.94	186	
Lower Bounds and Theta Notation	F15	33	358.72	160	< 0.0001
	S16	120	1090.02	861	
Algorithm Analysis Misunderstandings	F15	35	92.48	14	< 0.0001
	S16	140	462.57	324	
Problems, Algorithms, Programs	F15	34	203.52	21.5	< 0.0001
	S16	136	806.64	458	
Analyzing Problems	F15	34	78.23	20.5	< 0.0001
	S16	126	502.14	335.3	
Calculating Program Running time	F15	37	601.02	371	< 0.0001
	S16	118	1175.22	900	

It is clear from Table 1 that Spring 2016 (intervention) stu-

dents spent significantly more time interacting with AAVs than Fall 2015 (control) students spent reading the corresponding textual content for all OpenDSA sorting modules. Similarly, it is clear from Table 2 that Spring 2016 students spent significantly more time with the introductory algorithm analysis modules than Fall 2015 students did. Accordingly, from this analysis, we believe that students were more engaged with AAVs than with the textual content.

5.2 Student Satisfaction

In order to gauge student satisfaction, we offered a student survey at the end of the Spring 2016 semester. 90 students out of 155 responded. The survey solicited opinions about AAVs in terms of whether they helped with understanding the algorithm analysis concepts presented in the course. Students were also asked to provide suggestions to improve AAVs. In addition, they were asked whether algorithm analysis topics are easier for them as compared to algorithm dynamics concepts.

83% of the surveyed students reported that AAVs were useful to them in understanding the algorithm analysis concepts presented during the course. Of these, 51% reported that the AAVs were good as is and they did not suggest improvements, 15% did not suggest an improvement, but they did not mention clearly that they liked the AAVs as is, and 34% indicated that they found the AAVs useful, but also suggested ideas for improvement. Suggestions included tagging a specific slide to be able to view it later, adding more colors to the visual objects, and tying statements in the run-time analysis explanation to highlighted lines in the program, similar to the explanations for algorithm behavior. 8% of the surveyed students expressed negative feedback about AAVs, indicating that the visual depiction provided by AAVs did not help them to understand the concepts. 85% of the surveyed students stated that algorithm analysis topics are harder to them than algorithm dynamics. Of these, 26% attributed this to the presentation style of both topics in OpenDSA. They expressed that AVs do a better job in depicting algorithm dynamics concepts than AAVs do with algorithm analysis concepts.

5.3 Student Performance

It is almost certainly true that students were not learning the algorithm analysis material from the text-based presentation style (since they essentially did not read it). But it is possible that they learned this material elsewhere since only about 50% of the Fall 2015 group opened the material, as shown in Tables 1 and 2. Also, just because students spent more time looking at the AAVs does not necessarily mean that students learned more from them. However, Carroll's time on task hypothesis states that the longer a student spends engaging with the learning material, the more opportunities the student has to learn [3].

In order to evaluate AAVs in terms of student performance, we compared the control versus intervention group scores on the algorithm analysis part of the final exams. The final for both groups was divided into two sections, one for general DSA concepts and the other dedicated to algorithm analysis. Both groups were given two hours to complete the exam. Only similar items from the algorithm analysis section were used in our evaluation, for a total of 27 items. Table 3 shows the results from comparing the performance of the control against the intervention group using

a Mann-Whitney test. The results from the table are to the level of concepts addressed in the final exam. The addressed concepts are (1) relative growth rates, (2) algorithm upper, lower, and tight bounds, (3) problem upper and lower bounds, (4) best, average and worst cases, and (5) analyzing loop constructs. For concepts (1) and (5), we did not provide any AAV in OpenDSA. This gives us an opportunity to see the difference in performance on those concepts for which AAVs were provided versus those concepts where AAVs were not provided. Table 4 shows the effect sizes of the differences found in Table 3 using Cohen’s d for different sample sizes [10].

Table 3: Results from comparing the grades in the algorithm analysis part of the final from the control group (F15, $N = 67$) to the grades in the same part from the intervention group (S16, $N = 155$).

Concept	#Items	Mean		Median		p
		F15	S16	F15	S16	
All	27	16.07	19.97	16	20	< 0.0001
(1)	8	5.73	5.72	7	7	0.7470
(2)	7	3.13	4.76	2	5	< 0.0001
(3)	3	0.65	1.98	0	2	< 0.0001
(4)	5	3.44	4.38	4	5	< 0.0001
(5)	4	3.104	3.109	3	3	0.6697

Table 4: Effect sizes for the difference in grades between control and intervention students.

Concept	All	(1)	(2)	(3)	(4)	(5)
Effect Size	0.964	-0.005	0.892	1.239	1.012	0.005

In Table 3 we see that Spring 2016 students have significantly better scores on the whole set of questions than Fall 2015 students. For those concepts presented using AAVs, Spring 2016 students have better performance, while both groups have similar performance in those concepts not supported by AAVs. Further, the differences between the groups were large, with effect sizes of roughly one standard deviation in each instance for the concepts that were supported by AAVs.

6. THREATS TO VALIDITY

One threat to the validity of this study could be that we did not account for any difference in the students’ pre-knowledge between the control and intervention groups. We did not adopt a traditional pre/post-test approach for both groups. Offering a pre-test may not be affordable, as some instructors value the lecture time that a pre-test would displace. This was the case for the Fall 2015 instructor. We did conduct pre-tests for students during both Fall 2014 (identical content to Fall 2015) and Spring 2016 (intervention group). We found that the Fall 2014 and Spring 2016 groups had pre-test results that were not significantly different, based on a Mann-Whitney test ($p = 0.1727$). We would not expect there to be any significant difference in the pre-knowledge between any of the groups, as the students are taking the same course and they are all required to take similar pre-requisites (including a CS2 course). There were no other observed differences between the groups. Based on this, we do not believe that there is anything that may make

one group have more pre-knowledge than another group. We also performed a detailed comparison between the Fall 2015 and Spring 2016 groups based on demographic information such as gender, ethnicity, and whether the student is a first generation student. Results indicated no significant difference ($p > 0.05$) between the proportions of each demographic group between Fall 2015 and Spring 2016 students.

Since we gave a pre-test to the intervention group, but not the control group, a reasonable concern is that the act of taking the pre-test itself becomes an intervention that affects the final outcome. This appears not to be the case since, as described above, the intervention group’s scores on the final exam were significantly improved only for the three topics that were addressed by AAVs, and not for the two topics not addressed by AAVs. If taking the pre-test had affected performance on the final exam, then all topics would have been affected to some degree.

Another issue of concern is whether other factors caused the differences between the groups in the time spent on the algorithm analysis material or performance on the final exam. One factor could be that different instructors taught the course in Fall 2015 and Spring 2016. However, they used the same material and the same textbook (OpenDSA) for the course. The only difference was using AAVs in Spring 2016. Accordingly, we do not believe that this had a major contribution in the difference between both groups. Our belief here is supported by two points. First, our performance evaluation results indicate that students in Spring 2016 have better exam scores only on those concepts presented with AAVs. If there is any contribution by instructors, then we would expect a difference in the other concepts as well. Second, the Spring 2016 instructor also taught the course during Fall 2014. In Fall 2014, the algorithm analysis material was identical to Fall 2015 material (no AAVs). We have conducted an analysis similar to the one presented in Section 5.1. We compared the level of engagement of Fall 2014 students in the algorithm analysis material to the level of engagement of Fall 2015 students. This was done for OpenDSA sorting and algorithm analysis introductory modules. We found that for all modules there is no significant difference ($p > 0.05$) in the total time spent in the algorithm analysis material between Fall 2014 and Fall 2015 students. The only exception was for the “Problems, Algorithms, and Programs” module in which we found that Fall 2014 students spent significantly more time reading the algorithm analysis material presented in the module ($p = 0.0004$). This indicates that if there is any effect attributed to the instructors, it may be only for this module. This module presents the basic definitions of a problem, algorithm, and a program. These definitions were not tested in the final exam used for our performance evaluation. Accordingly, we don’t see an effect from this module on student scores. This indicates that AAVs are the major factor contributing to both greater student time spent and improved student performance.

7. CONCLUSIONS AND FUTURE WORK

AAVs were found to be more engaging in that intervention group students spent more time interacting with them than control group students spent with the corresponding textual discussion. The difference was significant for all OpenDSA modules containing AAVs. The majority of the surveyed students (83%) mentioned positive feedback about the ability of AAVs to help them understand the algorithm analysis

topics. However, a significant proportion of students gave at least one idea for improvement.

Given the engagement evidence from analyzing OpenDSA log data and the satisfaction evidence from the survey, we expected to see improved exam scores for the intervention group on the algorithm analysis part of the final exam. The scores for the intervention group students were significantly higher than the scores for the control group students, with large effect sizes of roughly one full standard deviation. Equally important, we also found that this improvement is only found on those questions addressing topics that were presented using AAVs. On other algorithm analysis questions that were not related to content presented using AAVs, there was no statistically significant difference between exam scores for the two groups. Accordingly, we believe that AAVs were a major contributor to this improvement.

Most students surveyed from the intervention group (85%) indicated that algorithm dynamics concepts are easier for them to understand than algorithm analysis concepts despite the use of AAVs. Control group students reported the same, in the same proportion. So AAVs did not change the students' opinion about the relative difficulty of algorithm analysis topics. Some students from Spring 2016 attributed the difficulty of algorithm analysis topics to the style of presentation. Even with AAVs, 26% of students still believe that the dynamics concepts are better presented than the analysis concepts in OpenDSA. As both are presented visually, this may indicate that students like AVs more than AAVs. One explanation for this is that visualizations appear better able to explain a dynamic evolving process as they help viewers to track patterns and observe relationships [2]. An algorithm is intrinsically a dynamic process that evolves over time and imposes some changes on a data structure. A run-time proof is not a dynamic process, but rather it is a sequence of statements that collectively validate or refute a given hypothesis.

We developed AAVs to visually present run-time proofs for the main sorting algorithms and the basic introductory concepts of algorithm analysis. Having proved to be effective in terms of student engagement and performance, we believe that more AAVs should be implemented and evaluated for other topics. In addition, students' suggestions for improving AAVs as indicated from the survey should be carefully studied and addressed. We believe that further evaluation studies should be done for AAVs, taking into consideration the differences that may arise from student demographics and majors. We encourage DSA educators to apply AAVs in their courses and evaluate their effectiveness.

8. ACKNOWLEDGMENTS

This work is supported by the National Science Foundation, under grants DUE-1139861 and IIS-1258471, and the VT-MENA program of Egypt. Special thanks to Eric Fouh.

9. REFERENCES

- [1] D. Blaheta. A visual proof of amortised-linear resizable arrays. *ACM SIGCSE Bulletin*, 41(3):338–338, 2009.
- [2] M. D. Byrne, R. Catrambone, and J. T. Stasko. Evaluating animations as student aids in learning computer algorithms. *Computers & Education*, 33(4):253–278, 1999.
- [3] J. Carroll. A model of school learning. *The Teachers College Record*, 64(8):723–723, 1963.
- [4] E. Fouh, D. Breakiron, S. Hamouda, M. Farghally, and C. Shaffer. Exploring students learning behavior with an interactive etextbook in computer science courses. *Computers in Human Behavior*, pages 478–485, December 2014.
- [5] E. Fouh, M. F. Farghally, S. Hamouda, K. H. Koh, and C. A. Shaffer. Investigating difficult topics in a data structures course using item response theory and logged data analysis. In *Proceedings of the 9th international conference on educational data mining*, pages 370–375, July 2016.
- [6] E. Fouh, V. Karavirta, D. A. Breakiron, S. Hamouda, S. Hall, T. L. Naps, and C. A. Shaffer. Design and architecture of an interactive etextbook—The OpenDSA system. *Science of Computer Programming*, 88:22–40, 2014.
- [7] M. T. Goodrich and R. Tamassia. Teaching the analysis of algorithms with visual proofs. In *ACM SIGCSE Bulletin*, volume 30, pages 207–211, 1998.
- [8] P. J. Guo, J. Kim, and R. Rubin. How video production affects student engagement: An empirical study of MOOC videos. In *Proceedings of the 1st ACM Conference on Learning@Scale*, pages 41–50, 2014.
- [9] R. H. Hammack and D. W. Lyons. Alternating series convergence: a visual proof. *Teaching Mathematics and its Applications*, 25(2):58, 2006.
- [10] L. Hedges and I. Olkin. Statistical methods for meta-analysis: Academic press. *Orlando, FL*, 1985.
- [11] C. Hundhausen, S. Douglas, and J. Stasko. A meta-study of algorithm visualization effectiveness. *Journal of Visual Languages and Computing*, 13:259–290, June 2002.
- [12] V. Karavirta and C. Shaffer. Creating engaging online learning material with the JSAV Javascript Algorithm Visualization Library. 9:171–183, April-June 2016.
- [13] H. B. Mann and D. R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *The annals of mathematical statistics*, pages 50–60, 1947.
- [14] R. Mayer. Applying the science of learning: Evidence-based principles for the design of multimedia instruction. *American Psychologist*, 63(8):760–769, 2008.
- [15] M. Parker and C. Lewis. What makes big-o analysis difficult: understanding how students understand runtime analysis. *Journal of Computing Sciences in Colleges*, 29(4):164–174, 2014.
- [16] C. A. Shaffer, M. L. Cooper, A. J. D. Alon, M. Akbar, M. Stewart, S. Ponce, and S. H. Edwards. Algorithm visualization: The state of the field. *ACM Transactions on Computing Education (TOCE)*, 10(3):9, 2010.
- [17] D. B. Sher. A visual proof for an average case of list searching. *ACM SIGCSE Bulletin*, 40(2):74–78, 2008.
- [18] H. Thompson and P. Chadhuri. An alternative visual analysis of the build heap algorithm. *ACM Inroads*, 2(3):31–32, 2011.